



**BITS Pilani**

# MFML AND MFDS

MFML and MFDS Team  
WILPD - CSIS



\* ZC416 Mathematical Foundations for  
Machine Learning / Data Science

Topic: # 7 Back Propagation & Automatic  
Differentiation



- In last lecture, we discussed about differentiation of univariate functions, partial differentiation, gradients and gradients of vector valued functions.
- Now, we will discuss back propagation and automatic differentiation.

# USEFUL IDENTITIES FOR COMPUTING GRADIENTS

- $\frac{\partial x^T a}{\partial x} = a^T$
- $\frac{\partial a^T x}{\partial x} = a^T$
- $\frac{\partial a^T x b}{\partial x} = ab^T$
- $\frac{\partial x^T B x}{\partial x} = x^T (B + B^T)$
- $\frac{\partial}{\partial s} (x - As)^T W (x - As) = -2(x - As)^T W A$

for symmetric  $W$ .

# BACKPROPAGATION AND AUTOMATIC DIFFERENTIATION



Consider the function

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2))$$

Taking derivatives

$$\begin{aligned}\frac{df}{dx} &= \frac{2x + 2x\exp(x^2)}{2\sqrt{x^2 + \exp(x^2)}} - \sin(x^2 + \exp(x^2))(2x + 2x\exp(x^2)) \\ &= 2x\left(\frac{1}{2\sqrt{x^2 + \exp(x^2)}} - \sin(x^2 + \exp(x^2))(1 + \exp(x^2))\right)\end{aligned}$$

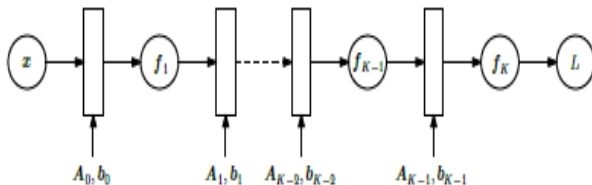


- The implementation of the gradient could be significantly more expensive than computing the function, which imposes unnecessary overhead where we get such lengthy expressions.
- We need an efficient way to compute the gradient of an error function with respect to the parameters of the model.
- For training deep neural network models, the backpropagation algorithm is one such method.

# BACKPROPAGATION AND AUTOMATIC DIFFERENTIATION



In neural networks with multiple layers



$$f_i(x_{i-1}) = \sigma(A_{i-1}x_{i-1} + b_{i-1})$$

where  $x_{i-1}$  is the output of layer  $i - 1$  and  $\sigma$  is an activation function.



To train these model, the gradient of the loss function  $L$  with respect to all model parameters  $\theta_j = \{A_j, b_j\}, j = 1, \dots, K$  and inputs of each layer needs to be computed. Consider,

$$f_0 := x$$

$$f_i := \sigma_i(A_{i-1}f_{i-1} + b_{i-1}), i = 1, \dots, K.$$

We have to find  $\theta_j = \{A_j, b_j\}, j = 1, \dots, K - 1$  such that

$$L(\theta) = ||y - f_K(\theta, x)||^2$$

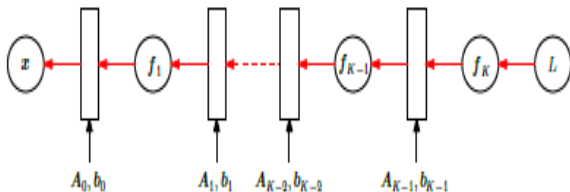
is minimum where  $\theta = \{A_0, b_0, \dots, A_{K-1}, b_{K-1}\}$



Using the chain rule, we get

$$\begin{aligned}\frac{\partial L}{\partial \theta_{K-1}} &= \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial \theta_{K-1}} \\ \frac{\partial L}{\partial \theta_{K-2}} &= \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \frac{\partial f_{K-1}}{\partial \theta_{K-2}} \\ \frac{\partial L}{\partial \theta_{K-3}} &= \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \frac{\partial f_{K-1}}{\partial f_{K-2}} \frac{\partial f_{K-2}}{\partial \theta_{K-3}} \\ \frac{\partial L}{\partial \theta_i} &= \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \dots \frac{\partial f_{i+2}}{\partial f_{i+1}} \frac{\partial f_{i+1}}{\partial \theta_i}\end{aligned}$$

# BACKPROPAGATION



If the partial derivatives  $\frac{\partial L}{\partial \theta_{i+1}}$  are computed, then the computation can be reused to compute  $\frac{\partial L}{\partial \theta_i}$ .

# EXAMPLE

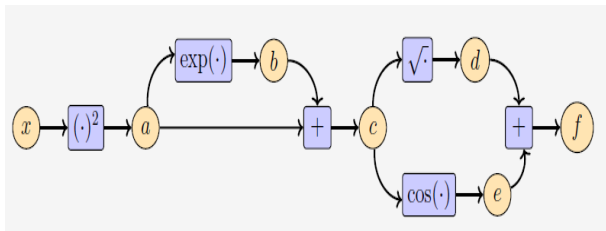


Consider the function

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2))$$

Let  $a = x^2$ ,  $b = \exp(a)$ ,  $c = a + b$ ,  $d = \sqrt{c}$ ,  $e = \cos(c)$

$\Rightarrow f = d + e$



# EXAMPLE



$$\begin{aligned}\Rightarrow \frac{\partial a}{\partial x} &= 2x \\ \frac{\partial b}{\partial a} &= \exp(a) \\ \frac{\partial c}{\partial a} &= 1 = \frac{\partial c}{\partial b} \\ \frac{\partial d}{\partial c} &= \frac{1}{2\sqrt{c}} \\ \frac{\partial e}{\partial c} &= -\sin(c) \\ \frac{\partial f}{\partial d} &= 1 = \frac{\partial f}{\partial e}\end{aligned}$$

# EXAMPLE



Thus, we have

$$\begin{aligned}\frac{\partial f}{\partial c} &= \frac{\partial f}{\partial d} \frac{\partial d}{\partial c} + \frac{\partial f}{\partial e} \frac{\partial e}{\partial c} \\ \frac{\partial f}{\partial b} &= \frac{\partial f}{\partial c} \frac{\partial c}{\partial b} \\ \frac{\partial f}{\partial a} &= \frac{\partial f}{\partial b} \frac{\partial b}{\partial a} + \frac{\partial f}{\partial c} \frac{\partial c}{\partial a} \\ \frac{\partial f}{\partial x} &= \frac{\partial f}{\partial a} \frac{\partial a}{\partial x}\end{aligned}$$

## EXAMPLE



Substituting the results, we get

$$\frac{\partial f}{\partial c} = \frac{1}{2\sqrt{c}} - \sin(c)$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c} \times 1$$

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial b} \exp(a) + \frac{\partial f}{\partial c} \times 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} 2x$$

Thus, the computation for calculating the derivative is of similar complexity as the computation of the function itself.

# FORMALIZATION OF AUTOMATIC DIFFERENTIATION



Let  $x_1, \dots, x_d$  : input variables.

$x_{d+1}, \dots, x_{D-1}$  : intermediate variables.

$x_D$  : output variable, then we have,

$$x_i = g_i(x_{Pa(x_i)})$$

Note that  $g_i$ s are elementary functions and are also called as forward propagation function and  $x_{Pa(x_i)}$  is the set of parent nodes of variable  $x_i$  in the graph.

# FORMALIZATION OF AUTOMATIC DIFFERENTIATION



Now,

$$f = x_D \Rightarrow \frac{\partial f}{\partial x_D} = 1$$

For other variables, using chain rule, we get

$$\frac{\partial f}{\partial x_i} = \sum_{x_j: x_i \in Pa(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i} = \sum_{x_j: x_i \in Pa(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial g_j}{\partial x_i}$$

The last equation is the back propagation of the gradient through the computation graph. For neural network training, we back propagate the error of the prediction with respect to the label.



# BACKPROPAGATION THROUGH A SIGMOID LAYER: A COMPLETE DERIVATION

Consider a standard neural-network layer

$$Z = Ax + b, \quad (1)$$

$$a = \sigma(Z), \quad (2)$$

where

- $x \in \mathbb{R}^n$  is the input column vector,
- $A \in \mathbb{R}^{m \times n}$  is the weight matrix,
- $b \in \mathbb{R}^m$  is the bias vector,
- $Z, a \in \mathbb{R}^m$  are the pre-activation and activation vectors,
- $\sigma(z) = \frac{1}{1+e^{-z}}$  is the sigmoid function.

Let the loss function be Loss, a scalar.

Define the backpropagated error signals

$$\delta^{(a)} = \frac{\partial \text{Loss}}{\partial a}, \quad (3)$$

$$\delta^{(z)} = \frac{\partial \text{Loss}}{\partial Z}. \quad (4)$$

For each coordinate,

$$a_i = \sigma(Z_i), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Thus,

$$\frac{\partial a_i}{\partial Z_i} = a_i(1 - a_i).$$

Since the sigmoid function is applied elementwise,

$$a_i = \sigma(Z_i),$$

so  $a_i$  depends only on  $Z_i$  and not on  $Z_j$  for  $j \neq i$ .

Therefore,

$$\frac{\partial a_i}{\partial Z_j} = 0 \quad \text{for } i \neq j.$$

The Jacobian matrix of  $a$  with respect to  $Z$  is

$$\frac{\partial \mathbf{a}}{\partial \mathbf{Z}} = \begin{bmatrix} \frac{\partial a_1}{\partial Z_1} & \frac{\partial a_1}{\partial Z_2} & \cdots & \frac{\partial a_1}{\partial Z_m} \\ \frac{\partial a_2}{\partial Z_1} & \frac{\partial a_2}{\partial Z_2} & \cdots & \frac{\partial a_2}{\partial Z_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial a_m}{\partial Z_1} & \frac{\partial a_m}{\partial Z_2} & \cdots & \frac{\partial a_m}{\partial Z_m} \end{bmatrix}.$$

Since  $\frac{\partial a_i}{\partial Z_j} = 0$  for  $i \neq j$  and  $\frac{\partial a_i}{\partial Z_i} = a_i(1 - a_i)$ , the Jacobian becomes

$$\frac{\partial \mathbf{a}}{\partial \mathbf{Z}} = \begin{bmatrix} a_1(1 - a_1) & 0 & \cdots & 0 \\ 0 & a_2(1 - a_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & a_m(1 - a_m) \end{bmatrix}.$$

In compact form,

$$\frac{\partial a}{\partial Z} = \text{diag}(a \odot (1 - a)).$$

Applying the chain rule componentwise:

$$\frac{\partial \text{Loss}}{\partial Z_i} = \frac{\partial \text{Loss}}{\partial a_i} \frac{\partial a_i}{\partial Z_i}.$$

**Remark:**  $\odot$  is the Hadamard product (also called the element-wise product) of two vectors of the same size is defined by multiplying corresponding entries.

Stacking these gives the vector relation:

$$\delta^{(Z)} = \delta^{(a)} \odot a \odot (1 - a).$$

We have

$$Z = Ax + b.$$

Each component satisfies

$$Z_i = \sum_{j=1}^n A_{ij}x_j + b_i,$$

and

$$\frac{\partial Z_i}{\partial A_{ij}} = x_j.$$

Thus by the chain rule:

$$\frac{\partial \text{Loss}}{\partial A_{ij}} = \frac{\partial \text{Loss}}{\partial Z_i} x_j.$$

In matrix form:

$$\frac{\partial \text{Loss}}{\partial A} = \delta^{(Z)} x^T.$$

The gradients with respect to the bias and input are:

$$\frac{\partial \text{Loss}}{\partial b} = \delta^{(z)},$$

$$\frac{\partial \text{Loss}}{\partial x} = A^T \delta^{(z)}.$$



Putting everything together:

$$\delta^{(Z)} = \delta^{(a)} \odot a \odot (1 - a), \quad (5)$$

$$\frac{\partial \text{Loss}}{\partial A} = \delta^{(Z)} x^T, \quad (6)$$

$$\frac{\partial \text{Loss}}{\partial b} = \delta^{(Z)}, \quad (7)$$

$$\frac{\partial \text{Loss}}{\partial x} = A^T \delta^{(Z)}. \quad (8)$$

These formulas constitute the complete backpropagation rules for a sigmoid neuron layer.



Thank you